

SIMATS ENGINEERING

ASSESSMENT TOOL 2 – ANSWER SCRIPT

Course Code:	CSA51	Course Name:	Cryptography and Network Security
Register No:	192521079	Course Outcome:	CO5 (BL5, BL6)

Annexure A – Pseudocode Writing Task (Algorithm Design)

Question 1 (5 Marks)

Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.

Problem Understanding

Inputs: client connection request, server's digital certificate (public key + CA signature). Outputs: an authenticated, encrypted channel between client and server. Constraints: server identity must be verified before any key exchange, and the session key must never be transmitted in plaintext.

BEGIN TLS_Handshake

```
// Step 1: Client Hello
CLIENT sends ClientHello(supported_ciphers, client_random)
      to SERVER

// Step 2: Server Hello + Certificate
SERVER selects cipher_suite from supported_ciphers
SERVER sends ServerHello(cipher_suite, server_random)
SERVER sends Certificate(server_public_key, CA_signature)

// Step 3: Certificate Verification (Authentication)
CLIENT verifies Certificate using trusted CA public key
IF signature_valid == FALSE OR certificate_expired == TRUE THEN
    ABORT handshake
    PRINT "Server authentication failed"
    RETURN
END IF

// Step 4: Session Key Generation
CLIENT generates pre_master_secret (random value)
CLIENT encrypts pre_master_secret using server_public_key
CLIENT sends EncryptedPreMasterSecret to SERVER

SERVER decrypts EncryptedPreMasterSecret using
      server_private_key

// Both sides independently derive the same session key
session_key = KDF(client_random, server_random,
      pre_master_secret)

// Step 5: Handshake Confirmation
CLIENT sends Finished(HMAC(session_key, handshake_log))
SERVER sends Finished(HMAC(session_key, handshake_log))

IF both Finished messages match THEN
    PRINT "TLS handshake successful"
ELSE
    ABORT handshake
END IF
```

```
// Step 6: Secure Data Exchange
WHILE connection_open == TRUE DO
    data = READ next application message
    ciphertext = ENCRYPT(data, session_key)
    SEND ciphertext over channel
    received = RECEIVE ciphertext
    plaintext = DECRYPT(received, session_key)
    PROCESS plaintext
END WHILE

END TLS_Handshake
```

Completeness & Justification

The pseudocode covers all four required stages -- client request, certificate-based server authentication with an explicit abort path on failure, symmetric session-key derivation via a key-derivation function (KDF), and a continuous encrypted data-exchange loop -- matching the real TLS handshake flow while remaining simplified for clarity.

Question 2 (5 Marks)

Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.

Problem Understanding

Input: an incoming packet (source IP, destination port, protocol) and an ordered rule table (each rule specifies IP/port/protocol patterns and an ALLOW/BLOCK action). Output: ALLOW or BLOCK decision, with a default-deny fallback if no rule matches.

```
BEGIN Firewall_Rule_Engine(packet, rule_table)

    // rule_table is an ordered list of rules:
    // {ip_pattern, port_pattern, protocol, action}

    FOR EACH rule IN rule_table DO

        ip_match  = MATCH(packet.source_ip, rule.ip_pattern)
        port_match = MATCH(packet.dest_port, rule.port_pattern)
        proto_match = (packet.protocol == rule.protocol
                       OR rule.protocol == "ANY")

        IF ip_match AND port_match AND proto_match THEN

            LOG(packet, rule.action, timestamp)

            IF rule.action == "ALLOW" THEN
                RETURN ALLOW
            ELSE
                RETURN BLOCK
            END IF

        END IF

    END FOR

    // Default policy: deny anything not explicitly allowed
    LOG(packet, "BLOCK (default-deny)", timestamp)
    RETURN BLOCK

END Firewall_Rule_Engine

// Driver loop
BEGIN Main
    WHILE network_interface HAS incoming_packet DO
        packet = CAPTURE next packet
        decision = Firewall_Rule_Engine(packet, rule_table)

        IF decision == ALLOW THEN
            FORWARD packet to destination
        ELSE
            DROP packet
        END IF
    END WHILE
END Main
```

Completeness & Justification

Rules are evaluated in order (first-match-wins, the standard firewall model), all three required attributes -- IP, port, protocol -- are checked together, every decision is logged for auditability, and a default-deny fallback ensures unmatched traffic is safely blocked rather than silently allowed.

Question 3 (5 Marks)

Write pseudocode to detect and classify malware behavior (virus, worm, or Trojan) based on propagation method, file modification patterns, and user interaction.

Problem Understanding

Input: observed behavior of a suspicious process (does it self-replicate over the network? does it modify or attach to files? did the user knowingly execute it? does it disguise itself as legitimate software?). Output: a classification label (VIRUS, WORM, TROJAN, or UNKNOWN) to drive the correct response.

```
BEGIN Classify_Malware(process)

    requires_user_action    = process.needs_user_execution
    spreads_over_network    = process.self_propagates_network
    modifies_other_files    = process.attaches_to_files
    disguised_as_legit      = process.appears_legitimate
    performs_hidden_action  = process.has_backdoor_or_payload

    // Rule 1: Worm -- spreads automatically, no user interaction
    IF spreads_over_network == TRUE
        AND requires_user_action == FALSE THEN
            RETURN "WORM"
        END IF

    // Rule 2: Virus -- attaches to files, needs a user to run
    //           the host file, does not self-propagate over network
    IF modifies_other_files == TRUE
        AND requires_user_action == TRUE
        AND spreads_over_network == FALSE THEN
            RETURN "VIRUS"
        END IF

    // Rule 3: Trojan -- disguised as legitimate, user knowingly
    //           runs it, but it performs a hidden malicious action
    IF disguised_as_legit == TRUE
        AND requires_user_action == TRUE
        AND performs_hidden_action == TRUE THEN
            RETURN "TROJAN"
        END IF

    // No confident match
    RETURN "UNKNOWN - flag for manual analysis"

END Classify_Malware

// Driver
BEGIN Main
    FOR EACH process IN running_processes DO
        IF process.is_suspicious == TRUE THEN
            label = Classify_Malware(process)
            LOG(process.id, label, timestamp)
            TRIGGER_RESPONSE(label, process)
        END IF
    END FOR
END Main
```

Completeness & Justification

The decision rules map directly to the textbook distinctions: worms are separated by autonomous network propagation without user action, viruses by file-attachment plus required user execution, and Trojans by disguise combined with a hidden payload; an UNKNOWN fallback ensures ambiguous cases are escalated rather than misclassified.

Question 4 (5 Marks)

Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.

Problem Understanding

Input: a stream of network packets/flows, a known-attack signature database, and a baseline traffic profile (normal behavior statistics). Output: an alert classification (KNOWN_ATTACK, ANOMALY, or NORMAL) for each flow, combining both detection techniques for higher accuracy.

```
BEGIN IDS_Hybrid_Detection(packet, signature_db, baseline)

    // ---- Stage 1: Signature-based detection ----
    FOR EACH signature IN signature_db DO
        IF MATCHES(packet.payload, signature.pattern) THEN
            RETURN {status: "KNOWN_ATTACK",
                    name: signature.attack_name,
                    confidence: HIGH}
        END IF
    END FOR

    // ---- Stage 2: Anomaly-based detection ----
    features = EXTRACT_FEATURES(packet)
    // e.g., packet rate, byte size, port entropy,
    // connection frequency

    deviation_score = COMPUTE_DEVIATION(features, baseline)
    // e.g., statistical distance or trained
    // ML model score

    IF deviation_score > ANOMALY_THRESHOLD THEN
        RETURN {status: "ANOMALY",
                score: deviation_score,
                confidence: MEDIUM}
    END IF

    RETURN {status: "NORMAL", confidence: HIGH}

END IDS_Hybrid_Detection

// Driver loop
BEGIN Main
    WHILE traffic_stream HAS next_packet DO
        packet = CAPTURE next_packet
        result = IDS_Hybrid_Detection(packet, signature_db,
                                      baseline)

        IF result.status != "NORMAL" THEN
            LOG(packet, result, timestamp)
            RAISE_ALERT(result)
        END IF

        // Continuously refine baseline with confirmed-normal
        // traffic to reduce future false positives
        IF result.status == "NORMAL" THEN
            UPDATE_BASELINE(baseline, features_of(packet))
        END IF
    END WHILE
END Main
```

Completeness & Justification

Signature matching runs first for fast, low-false-positive detection of known attacks; anomaly detection then catches novel/zero-day threats that have no signature. Continuous baseline updating from confirmed normal traffic keeps the anomaly detector accurate over time, addressing the accuracy/false-positive trade-off central to real IDS design.

Question 5 (5 Marks)

Create pseudocode for an Intrusion Prevention System (IPS) that monitors network traffic in real time and automatically blocks malicious requests while logging intrusion attempts.

Problem Understanding

Input: a real-time packet/flow stream and a detection engine (as in Q4). Output: an active response -- malicious traffic must be blocked inline (not just alerted on), and every intrusion attempt must be logged for forensic review, while legitimate traffic passes through with minimal delay.

```
BEGIN IPS_RealTime_Monitor(traffic_stream, detection_engine,
                           blocklist)

    WHILE traffic_stream HAS next_packet DO

        packet = CAPTURE next_packet    // inline, in the data path

        // Fast-path: check active blocklist first
        IF packet.source_ip IN blocklist THEN
            DROP packet
            LOG(packet, "BLOCKED - blacklisted IP", timestamp)
            CONTINUE
        END IF

        result = detection_engine.ANALYZE(packet)
                // reuses signature + anomaly detection

        IF result.status == "KNOWN_ATTACK"
            OR (result.status == "ANOMALY"
                AND result.score > BLOCK_THRESHOLD) THEN

            DROP packet
            ADD packet.source_ip TO blocklist
                WITH expiry = BLOCK_DURATION
            LOG(packet, result, timestamp, action="BLOCKED")
            NOTIFY_ADMIN(result)

        ELSE IF result.status == "ANOMALY" THEN
            // Suspicious but below the auto-block threshold
            FORWARD packet
            LOG(packet, result, timestamp, action="MONITORED")

        ELSE
            FORWARD packet    // normal traffic, minimal delay

        END IF

    END WHILE

END IPS_RealTime_Monitor

// Periodic maintenance
BEGIN Cleanup_Blocklist(blocklist)
    FOR EACH entry IN blocklist DO
        IF CURRENT_TIME > entry.expiry THEN
            REMOVE entry FROM blocklist
        END IF
    END FOR
END Cleanup_Blocklist
```

Completeness & Justification

Unlike an IDS, this engine sits inline and takes action: confirmed attacks and high-confidence anomalies are dropped immediately and the offending IP is temporarily auto-blocklisted, medium-confidence anomalies are forwarded but monitored (avoiding over-blocking legitimate users), every event is logged for audit, and a blocklist-cleanup routine prevents stale entries from permanently denying service, keeping the design both protective and efficient in real time.

--- End of Answer Script ---